

SIH26023 — AI-Powered Geological, Mining & Reporting Solution for CMPDI / CIL Subsidiaries

Ministry of Coal

Dept: Coal India Limited

Category: Software

Theme: Smart Automation

Deadline: 30 Sept 2026

Contents

1. The problem, in plain language
2. Who is actually asking, and why it's hard
3. The three things you are literally asked to build
4. What "winning" is measured by (the hidden rubric)
5. Reference architecture + data flow
6. Tech stack mapped to what you already know
7. Where n8n fits (orchestration split)
8. MVP scope for the hackathon vs. the full vision
9. Traps that sink teams on this exact PS
10. How to actually stand out
11. Reference material & tools

1. The problem, in plain language

Strip away the government phrasing and this is what's happening: CMPDI (Central Mine Planning & Design Institute, CIL's engineering/planning

arm) and other CIL subsidiaries have to answer questions constantly — from Parliament, from ministries, from internal management — about things like coal reserves, mine geology, production figures, and safety data. The answers to those questions live scattered across:

- Scanned old geological survey PDFs (some possibly decades old, poor scan quality)
- Digital-native reports and spreadsheets
- Images (maps, charts, seismic/geological diagrams)
- Historical archives with no consistent structure

Right now, a human has to manually dig through all of this, cross-check it, and compile a report — which is slow, error-prone, and depends entirely on which specific employee happens to know where things are. You are being asked to replace that manual compilation process with an AI system that can ingest all these messy sources, understand them, and produce structured, trustworthy reports and answers on demand.

One-sentence problem statement you can say out loud in a pitch:

"We're building an AI platform that turns CIL's scattered geological and mining documents — scanned or digital — into instantly queryable, auto-verified reports, cutting report preparation time from days to minutes while keeping every number traceable back to its source document."

2. Who is actually asking, and why it's hard

This isn't a generic "build a chatbot for PDFs" problem — three things make it genuinely hard, and judges from a PSU like CIL will specifically probe these:

Heterogeneous, messy inputs

Scanned PDFs (possibly skewed/low-DPI), born-digital

Accuracy is non-negotiable

These numbers feed Parliamentary answers. A

PDFs, Word/Excel files, images, and "historical archives" — likely inconsistent formats across decades. A single OCR pipeline will not be enough; you need a routing layer that decides how to process each document type.

hallucinated production figure is a real-world governance failure, not just a bad demo. Every claim needs traceability — a citation back to the exact page/paragraph it came from.

Domain vocabulary

Mining/geological terminology (seam thickness, overburden ratio, strata classification, gradation, reserve categories per UNFC) won't be well represented in a generic embedding model without some domain grounding.

Institutional scale & phased rollout

The PS explicitly expects phased delivery — requirement analysis → digitization → platform → testing → integration → training → continuous improvement. Judges will want to see you understand this isn't a weekend toy, even if your demo is small.

3. The three things you are literally asked to build

The PS names exactly three deliverables. Build your entire architecture around these three, don't invent a fourth thing that dilutes focus.

Module 1 — Automated Report Generation Platform

- Ingests documents (scanned + digital) → extracts structured data (figures, tables, dates, locations, entities) → validates/cross-checks against other sources → assembles a formatted report (e.g. auto-filled template: production summary, geological survey abstract, safety incident report).

- Needs a **human-in-the-loop review step** before final export — judges will ask "what stops it publishing a wrong number to Parliament?"

Module 2 — Automated Word Cloud & Topic Identification Module

- This is a lighter NLP task relative to the others — don't over-invest time here, but don't skip it either since it's explicitly named.
- Given a corpus (e.g. all reports from a subsidiary in a quarter), extract dominant topics/themes — useful for spotting patterns like recurring safety issues or frequently mentioned geological zones.
- Techniques: TF-IDF + word cloud for a quick visual; BERTopic or LDA for actual topic modeling if you want depth; an LLM-based "cluster and label" pass is the fastest way to get high-quality topic names for a demo.

Module 3 — AI-Based Query & Response System

- This is your core RAG system — natural-language question in, grounded and cited answer out. E.g. "What was the total reserve estimate for the Jharia coalfield in the 2019 survey?"
- Must support cross-document synthesis (an answer might require combining a 1998 scanned survey with a 2023 digital report) and must cite sources.

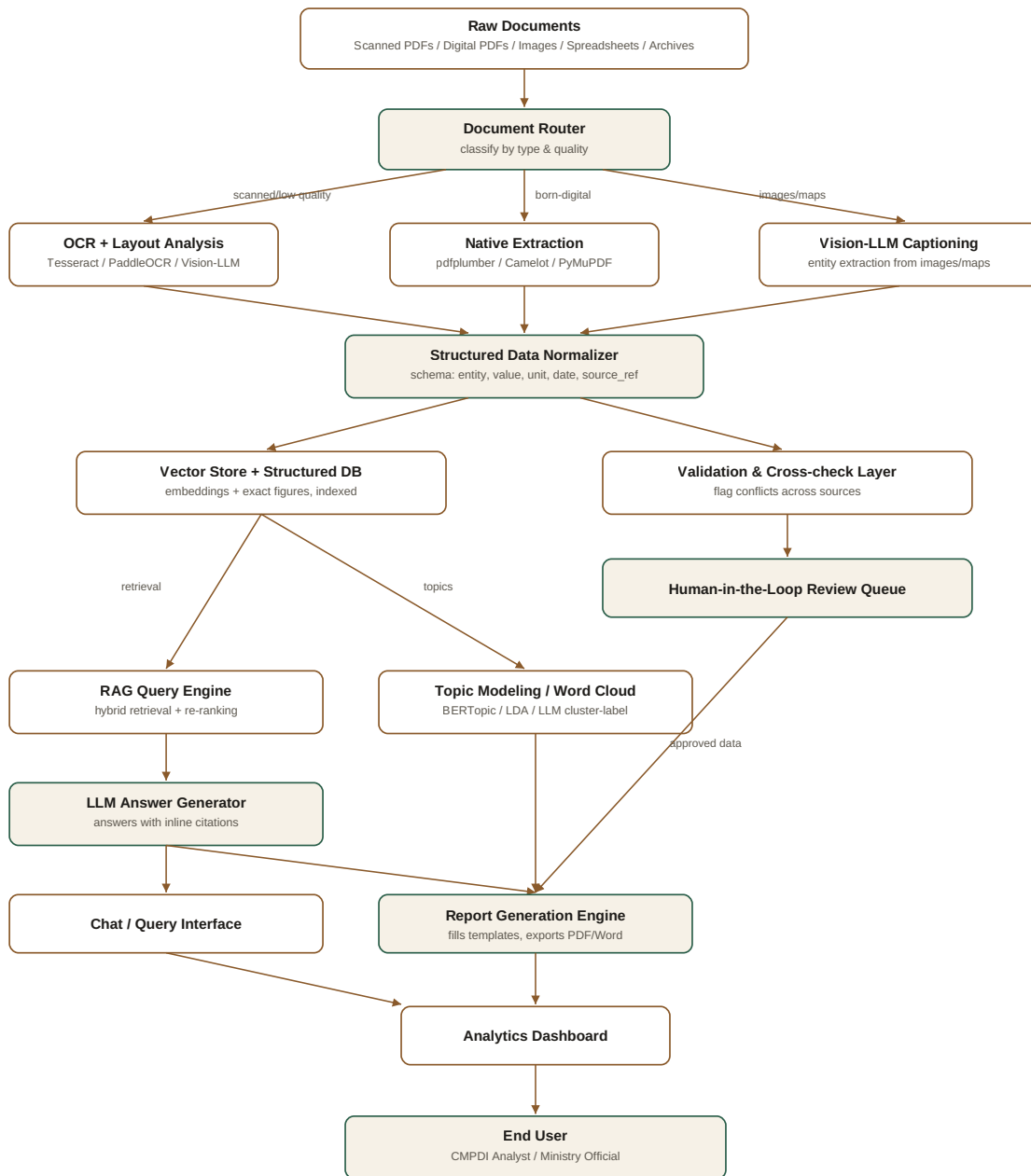
How the three modules connect: Module 1's ingestion/extraction pipeline is the shared foundation. Module 3 (query system) sits on top of the same indexed data. Module 2 (word cloud/topics) is a byproduct view over the same corpus. Build the ingestion pipeline once, well — it's the backbone for all three asks.

4. What "winning" is measured by (the hidden rubric)

The PS gives you the exact metrics it wants quantified — treat these as your evaluation criteria, and design your demo to visibly report numbers against each one:

Stated expected benefit	What you should actually measure/show
Reduction in report preparation time	Time a human takes to manually find+compile an answer vs. your system's response time. Show both on screen.
Maximum accuracy in extraction/report generation	Precision/recall on a small labeled test set of extracted fields (even 20-30 documents you've manually annotated is enough for a hackathon).
Maximum automation of repetitive workflows	% of the pipeline that runs with zero human touch vs. requiring review — be honest, show the human-in-loop step, judges respect this more than a false "100% automated" claim.
Faster response to high-level/parliamentary inquiries	Query latency + a worked example of a "parliamentary-style" question answered end-to-end.
Data accessibility, transparency, standardization	A unified schema you normalize all extracted data into, regardless of source format.
Traceability	Every generated number/sentence links back to a source document + page/paragraph. This is your single most important non-functional requirement — build it in from day one, not as an afterthought.

5. Reference architecture & data flow



Orchestration note: the triggers, routing, scheduling, and human-review-notification arrows in this diagram (ingestion trigger, document

router, human-in-the-loop queue, scheduled report generation) map directly onto n8n workflows. The extraction, validation, and RAG/query boxes stay as custom FastAPI + LangChain code. See [section 7](#) for the full split.

Flow explained, stage by stage

1. **Ingestion & routing** — every document is classified first (scanned vs. digital, text-heavy vs. table-heavy vs. image) so you don't run expensive OCR on documents that don't need it.
2. **Extraction** — OCR for scans, native parsing for digital files, vision-LLM for maps/diagrams. Output is always normalized into one common schema (entity, value, unit, date, source document, page number).
3. **Storage** — normalized structured facts go into a relational/structured store (for exact figures), and the full text chunks go into a vector store (for semantic search).
4. **Validation** — a rules + LLM layer cross-checks new extractions against existing records and flags contradictions for human review rather than silently overwriting.
5. **Query system** — hybrid retrieval (keyword + semantic) over the vector store, re-ranked, then passed to an LLM that must answer only with citations back to `source_ref`.
6. **Report generation** — takes either a query result or a scheduled request ("generate Q3 production report for X subsidiary") and fills a structured report template.
7. **Topic/word-cloud module** — runs periodically or on-demand over a document set, independent of the live query path.

6. Tech stack mapped to what you already know

Your existing stack (LangChain, FastAPI, RAG pipelines, multi-agent systems, Vertex AI, AWS, React/Next.js) maps onto this almost directly —

you don't need to learn a new paradigm, just apply it to messier, domain-specific documents.

Layer	Suggested tools	Notes
OCR / extraction	PaddleOCR or docTR (open-source, no API cost) or a Vision-LLM (Gemini via Vertex AI, since you already use it) for messy scans	Vision-LLM OCR is often the fastest way to get clean markdown/table output from ugly scans — worth prototyping first for a hackathon timeline
Table/native PDF parsing	pdfplumber, Camelot, PyMuPDF	Free, fast, good for born-digital reports
Structured storage	Postgres (facts/entities)	Keep exact figures in structured rows, not just embeddings — needed for "maximum accuracy" claims
Vector store	pgvector, Qdrant, or FAISS for a hackathon-scale demo	pgvector keeps your infra simple if you're already on Postgres
RAG orchestration	LangChain (you already use this) — hybrid retriever (BM25 + embeddings) + re-ranker	Consider a "generate hypothetical answer first, then retrieve" step — this is a technique that's shown strong results in recent RAG competitions
LLM	Vertex AI (Gemini) or any hosted LLM with strong citation-following behavior	Prompt it explicitly to refuse answering if no supporting source is retrieved — this is your accuracy safety net
Multi-agent orchestration	Your existing multi-agent experience fits well for: one agent for extraction QA, one for cross-source conflict resolution, one for report drafting	Don't over-engineer this for the hackathon demo — a clean pipeline beats an over-complicated agent swarm on stage
Backend	FastAPI	Matches your existing stack
Frontend		

	React/Next.js dashboard: chat interface + report viewer + topic/word-cloud analytics view	Judges respond well to a clean, working demo UI over a slide describing one
Topic modeling	BERTopic (good balance of quality/speed) or an LLM clustering pass for the word cloud module	This module is lower priority — timebox it

7. Where n8n fits (orchestration split)

n8n is a strong fit as the **orchestration and glue layer** around this system — not as the home for your core RAG/extraction logic. This split also happens to mirror the PS's own "structured phases" language, which is a nice narrative hook for your pitch.

Put in n8n

- Ingestion triggers — watch a folder/Drive/upload endpoint for new documents and kick off the pipeline
- Routing — branch scanned vs. digital vs. image documents to the right processing step
- Calling external services as nodes — OCR API, LLM API, vector DB upsert
- Scheduled report generation — e.g. "every Monday, compile the week's extracted data into a report"
- Human-in-the-loop steps — flag a conflict → notify a

Keep as custom code (FastAPI + LangChain)

- Hybrid retrieval, re-ranking, and citation-tracking logic — too fine-grained to control precisely through nodes
- The normalized extraction schema and validation/conflict-detection logic
- Vector search at real scale — call your vector store directly rather than wrapping it thinly in n8n
- The core LLM prompting/answer-generation logic where citation-following behavior needs tight control

reviewer (Slack/email) →
wait for approval → continue
the flow

- Phased-rollout story on slides — n8n's visual flow doubles as a clean diagram of your pipeline for judges

Demo framing risk: a live judged demo built mostly in n8n can read as "no-code automation" rather than "AI engineering," which risks underselling the technical depth expected on a Software/Smart Automation PS at this level. Lead the technical Q&A with your custom RAG/extraction code, and use n8n visibly for the orchestration/phased-pipeline story where it actually strengthens your narrative.

8. MVP scope for the hackathon vs. the full vision

Build for the demo (MVP)

- Ingest ~20-30 sample documents (mix of scanned + digital — you can construct/simulate realistic mining report samples if real CIL data isn't available)
- One working extraction pipeline with the normalized schema
- A functioning RAG query interface with visible citations

Describe as the full vision (slides only)

- The phased rollout plan (requirement analysis → digitization → integration → training → continuous enhancement) — the PS literally gives you this phase list, reuse it
- Role-based access control for different CIL subsidiaries
- Integration with CIL's existing IT systems

- | | |
|---|---|
| <ul style="list-style-type: none">• One auto-generated report from a template, with numbers traced to source• A word cloud/topic view over the sample corpus• A dashboard showing your "time saved" and "accuracy" metrics from section 4 | <ul style="list-style-type: none">• Multilingual support (Hindi + regional languages) for wider adoption• An audit trail / version history for regulatory compliance |
|---|---|

9. Traps that sink teams on this exact problem statement

- **Demoing only on clean digital PDFs.** Judges will specifically ask "what about the scanned ones?" — always include at least one genuinely poor-quality scanned sample in your live demo.
- **No traceability.** If your system gives an answer with no visible link back to a source document/page, you will get flagged immediately on a government-reporting use case.
- **Ignoring conflicting sources.** Real archives will have inconsistent numbers across years/documents. Showing that your system flags a conflict rather than silently picking one builds massive credibility.
- **Treating this as a generic "chat with your PDF" app.** This PS explicitly wants three named deliverables (report generation, word cloud/topics, query system) plus phased implementation thinking — a generic RAG chatbot demo without those pieces reads as under-scoped.
- **No security/access story.** Coal reserve and production data can be sensitive; even a one-slide mention of role-based access and data governance shows maturity.

10. How to actually stand out

- **Lead with the validation/conflict-detection layer** — most teams will build extraction + RAG and stop there. A visible "here are two documents disagreeing on this reserve figure, flagged for human review" demo moment is memorable and directly answers the PS's stated pain point ("high dependence on individual expertise," "manual errors").
- **Quantify everything on stage** — literally show a counter: "Manual process: ~45 min per query (estimated). Our system: 8 seconds. Extraction accuracy on test set: 91%." Judges reward teams who measure against the PS's own stated success metrics.
- **Show the phased rollout plan as a real Gantt-style slide**, mapped to the PS's own phase list — this signals you understand it's a government deployment, not a toy app.
- **Domain-tune your retrieval** — even a small glossary of mining/geological terms (seam, overburden, reserve categories) fed into your retrieval/prompt step will visibly outperform a generic RAG setup on domain questions, and you can show that comparison live.

11. Reference material & tools worth exploring

- **Similar past/parallel SIH problem statements** to study for structure — SIH 2025 had a closely related PS (#25231): a multimodal RAG system ingesting DOC/PDF/images/audio for law enforcement reports, with citation transparency as a named requirement — worth searching for public repos/write-ups from teams that attempted it, since the architecture overlaps heavily with yours.
- **Open-source document extraction**: docTR, PaddleOCR, deepdoctection (layout + table + OCR pipeline in one), Docling (IBM, good for born-digital PDFs to Markdown), img2table (table extraction via OpenCV).
- **Open-source RAG engines for reference architecture**: RAGFlow (deep-document-understanding RAG engine, good architectural reference even if you build your own).

- **RAG technique:** "hypothetical answer generation before retrieval" (generate a draft answer first, use it to improve retrieval) — a technique that won a recent SIGIR RAG challenge and is straightforward to bolt onto a LangChain pipeline.
- **Topic modeling:** BERTopic documentation, for a clean topics-over-time view that would work well for "Ministry of Coal quarterly themes."
- **Domain context:** look up CMPDI's public role/mandate and CIL's UNFC reserve classification system — even surface-level familiarity with these terms in your pitch will differentiate you from teams treating this as a generic document problem.

Built as a working reference for SIH26023 prep — treat the MVP scope and metrics table as your actual build checklist, and the vision section as slide material only.